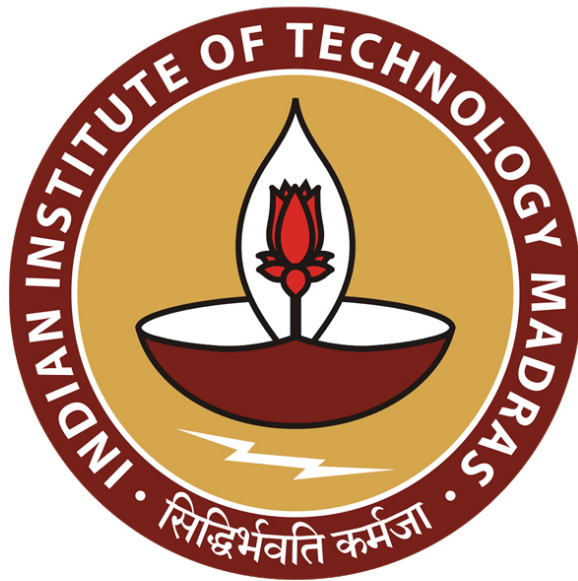




IIT Madras
ONLINE DEGREE

Database Management Systems
Professor Partha Pratim Das
Department of Computer Science & Engineering
Indian Institute of Technology, Kharagpur
Entity-Relationship Model/3

(Refer Slide Time: 00:22)

The screenshot shows the IIT Madras BSc Degree online course interface. The top header is blue with the IIT Madras logo and text. The left sidebar is blue with a list of topics. The main content area is white with a blue header for the module. The text in the main area includes the module title, the professor's name, department, and email address.

IIT Madras
BSc Degree

Module 20
Partha Pratim Das

Objectives & Outline
ER Features
Notation
Specialization
Generalization
Aggregation
Design Issues
Entity vs. Attribute
Entity vs. Relationship
Binary vs. Non-Binary
Design Decisions

Database Management Systems
Module 20: Entity-Relationship Model/3

Partha Pratim Das
Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur
ppd@cse.iitkgp.ac.in

Welcome to Module 20 of Database Management Systems course in the IIT Madras online B.Sc. program. We have been discussing logical database design, conceptual design issues using entity relationship model.

(Refer Slide Time: 00:34)

The screenshot shows the IIT Madras BSc Degree online course interface for the Module Recap. The top header is blue with the IIT Madras logo and text. The left sidebar is blue with a list of topics. The main content area is white with a blue header for the module. The text in the main area includes the module title and a list of topics.

IIT Madras
BSc Degree

Module 20
Partha Pratim Das

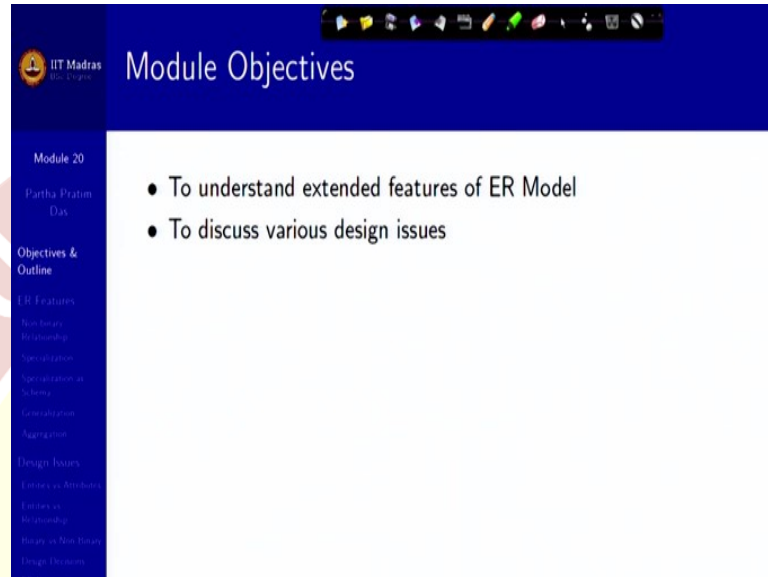
Objectives & Outline
ER Features
Notation
Specialization
Generalization
Aggregation
Design Issues
Entity vs. Attribute
Entity vs. Relationship
Binary vs. Non-Binary
Design Decisions

Module Recap

- ER Diagram for ER Models
- Translation of ER Models to Relational Schema

And in the last module we have discussed the ER diagram representation and how simple ER diagrams, ER models can be translated to relational schema.

(Refer Slide Time: 00:48)



Module Objectives

- To understand extended features of ER Model
- To discuss various design issues

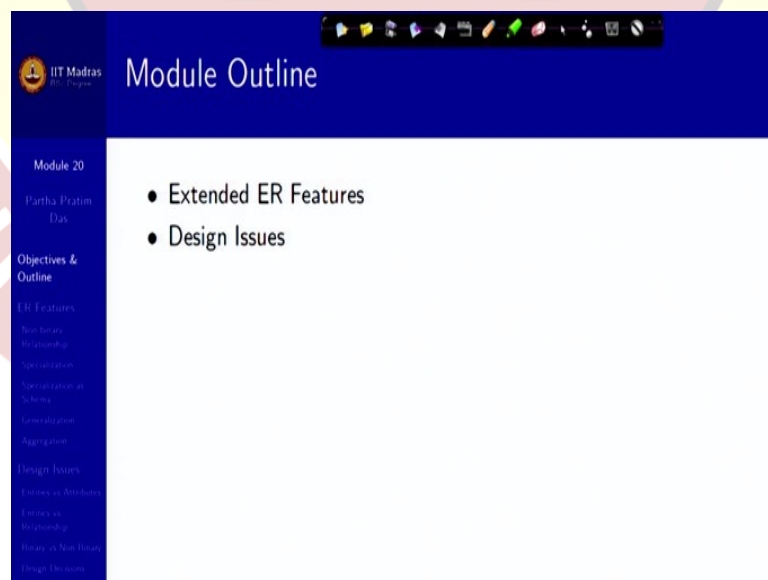
Module 20
Partha Pratim Das

Objectives & Outline

- ER Features
 - Relationships
 - Relationships
 - Specialization
 - Generalization or Schema
 - Generalization
 - Aggregation
- Design Issues
 - Entity vs. Relationship
 - Entity vs. Relationship
 - Entity vs. Relationship
 - Entity vs. Relationship
 - Entity vs. Relationship
 - Entity vs. Relationship

In this concluding module on the topic, we will discuss some more extended features of the ER model and give a glimpse of some of the design issues and considerations.

(Refer Slide Time: 01:04)



Module Outline

- Extended ER Features
- Design Issues

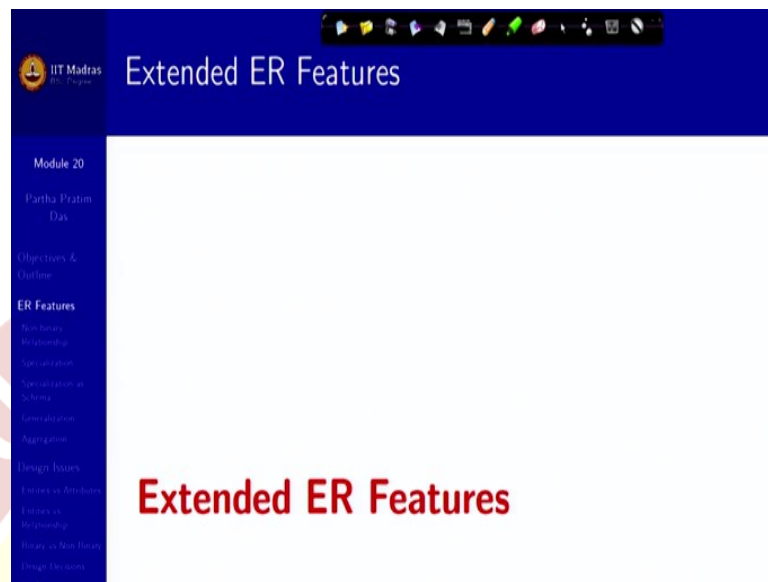
Module 20
Partha Pratim Das

Objectives & Outline

- ER Features
 - Relationships
 - Relationships
 - Specialization
 - Generalization or Schema
 - Generalization
 - Aggregation
- Design Issues
 - Entity vs. Relationship
 - Entity vs. Relationship
 - Entity vs. Relationship
 - Entity vs. Relationship
 - Entity vs. Relationship
 - Entity vs. Relationship

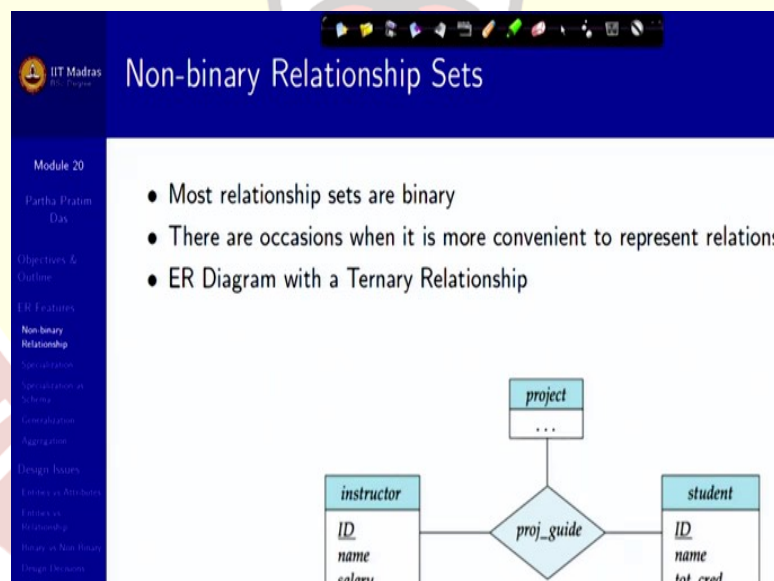
So, these are the two things to discuss primarily.

(Refer Slide Time: 01:08)



So, extended ER features, we will discuss a couple of them.

(Refer Slide Time: 01:10)



The first is, we have mentioned already, that relationship sets are mostly binary, but at times it may be convenient to represent a relationship as non-binary, like here is a ternary relationship between project, instructor and student. So, it says that for a student, what is a project and who is the instructor. This is a triplet combination which is kept in this relationship which is ternary.

(Refer Slide Time: 01:46)



IIT Madras
M.Tech. Degree

Cardinality Constraints on Ternary Relationship

Module 20

Partha Pratim Das

Objectives & Outline

ER Features

Non-binary Relationship

Specialization

Generalization

Aggregation

Design Issues

Entity vs. Attribute

Entity vs. Relationship

Design in New Paradigm

Design Decisions

- We allow at most one arrow out of a ternary (or greater degree) relationship to have a cardinality constraint
- For example, an arrow from *proj_guide* to *instructor* indicates each student guide for a project
- If there is more than one arrow, there are two ways of defining the meaning
 - For example, a ternary relationship *R* between *A*, *B* and *C* with a cardinality constraint could mean
 - a) Each *A* entity is associated with a unique entity from *B* and *C*
 - b) Each pair of entities from (*A*, *B*) is associated with a unique entity from *C*
 - Each alternative has been used in different formalisms



IIT Madras
M.Tech. Degree

Non-binary Relationship Sets

Module 20

Partha Pratim Das

Objectives & Outline

ER Features

Non-binary Relationship

Specialization

Generalization

Aggregation

Design Issues

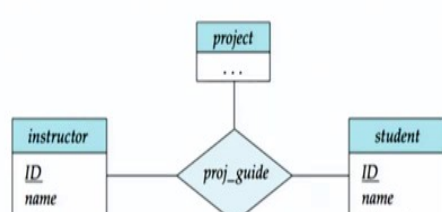
Entity vs. Attribute

Entity vs. Relationship

Design in New Paradigm

Design Decisions

- Most relationship sets are binary
- There are occasions when it is more convenient to represent relations as non-binary
- ER Diagram with a Ternary Relationship



Now, there are a couple of questions here. One is the question is what could be the cardinality constraint? For example, if I go back to the previous say, if I say this, then what does that mean? That means, there can be only one instructor. That is a uniqueness. Or if I say that, which means only one student can work in the project and so on. So, the question is, if we say what does it mean, which one is one or which combination is one, so that is the basic problem.

So, if you look in a little abstract terms, then if I have a ternary relationship between *A*, *B* and *C* with arrows to *B* and *C*, so I have *A*, I have *B*, I have *C* and I have *R*, so this, this and this. Now, if we have arrows to *B* and *C* like this, what does that mean? That each entity *A* is associated with a unique entity from *B* and *C*, could be one meaning. Other meaning could be that each pair

of entity A, B is associated with a unique entity C or and each pair of A, C is associated with a unique entity in B. You can give any of these interpretations.

So, there is some kind of ambiguity when you have more than one arrows. And in different formalisms through the development time, all these alternatives have been used by some one or the other, but it creates confusion. So, we will outlaw the use of more than one arrow if it is a non-binary relationship. So, this is a ground rule to follow.

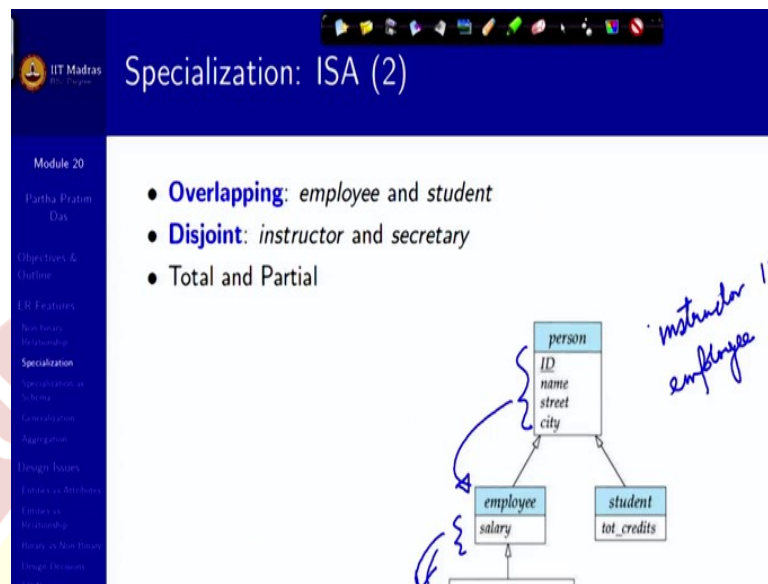
(Refer Slide Time: 04:37)

The screenshot shows a presentation slide from IIT Madras. The title is 'Specialization: ISA'. The slide content includes a list of bullet points:

- **Top-down design process:** We designate sub-groupings within an entity set that are distinctive from other entities in the set
- These sub-groupings become lower-level entity sets that have attributes in relationships that do not apply to the higher-level entity set
- Depicted by a *triangle* component labeled ISA (e.g., *instructor* "is a" *professor*)
- **Attribute inheritance:** A lower-level entity set inherits all the attributes and relationship participation of the higher-level entity set to which it is linked

Next, let us talk about ISA relationship. I am sure most of you have heard about ISA or specialization generalization relationship. So, this happens is when there is, you have an entity set, so if we look at top down, we have an entity set and from that entity set, we are, we have a specialized low level entity set that may have additional attributes and or that may be participating in relationships in which the higher level generalized entity set does not participate. So, we typically show this by a triangle component in the ISA and attributes have an inheritance.

(Refer Slide Time: 05:35)



So, let me go to the example it will become clearer. So, this is kind of the highest level or the root entity set, the most generalized. Then what is say by this arrow I mean ISA. So, we will read and textually write this in this way student ISA person. So, which means that student will have all the attributes inherited from person, from higher to lower you will have all the attributes inherited. Similarly, employee is another specialization which has all the attributes inherited from person, but in addition it has its own attribute salary, student has its own attribute total credits.

Now, if you look at instructor, so instructor ISA employee, employee ISA person. So, the instructor inherits all attributes of person which is already there in employee and the additional attribute of employee, instructor will inherit all of those and it adds one of its own which is rank, similarly, for the secretary and so, on.

So, this is how the hierarchy works. And it is important in multiple ways; one is it is giving a different form of identifying notion to the entities. And obviously, employees will engage in variety of relationships where students have no role. And students will engage in a variety of relationships where employees may not have a role, whereas in general, a person will engage in some general activities.

So, there, when you do specialization, there could be couple of different situations; one is overlapping, which is kind of inclusive situation. For example, employee and student is

overlapping. That is an employee can also be a student or a student can also be an employee. So, it is possible that some person entity exists in both. But instructor and secretary are disjoint. No instructor can be secretary or no secretary can be instructor. So, an employee cannot be both an instructor and a secretary.

Then, the question is whether this inheritance is total or partial. That is given any generalized set is it necessary that all entities in that set exist in at least one specialized entity set, which if I read in terms of person here says is it necessary that every person has to be either an employee or a student or both or could I have a person who is neither an employee nor a student just a person. So, if it is allowed, if my design allows that it is not necessary for every entity in the higher entity set to be necessarily be a member of one or more lower entity set, if it is not mandatory, then the inheritance or specialization is partial. If it is mandatory, it is total.

So, this is kind of the notional modeling. So, this notion exists very strongly in several domains, particularly if you have done any object oriented analysis design, you have used Java, C++, you must have come across this specialization, generalization class, subclass, parent class, child class concept. So, it is just a parallel of that. And this is the representation of that information in the ER model.

(Refer Slide Time: 10:21)

Representing Specialization via Schema

- Method 1:
 - Form a schema for the higher-level entity
 - Form a schema for each lower-level entity set, include primary key entity set and local attributes

schema	attributes
person	ID, name, street, city
student	ID, tot_cred
employee	ID, salary

- Drawback: Getting information about, an *employee* requires access to the one corresponding to the low-level schema and the one corresponding to the high-level schema

So, the question is, again, the related question, how do we represent it in the relational schema. There could be two ways, of course. One is, I represent the higher level entity and for the lower

level entity which is specialization, I use the key of the higher level entity, so that I can anytime get the other attributes of the higher level entity and put the attribute of the lower level entity. So, what we are doing is person is at the route, so I describe everything for the person. For employee, I just use ID, which is basically the ID coming from the person and the total credit, which is additional one, similarly, for the employees. So, this could be one way.

So, the advantage here is you do not have the problem of redundancy, anomaly all these. But it is the flip side is, whenever I want to get, say, for students' total credit, naturally, most often, I would also need to know the name. So, any reporting will need the name. So, to get the name, I have to go to another entity set, which means that I have to go to another table, which means that I have to do a join and so on, so expensive.

(Refer Slide Time: 11:46)

Representing Specialization as Schema (2)

- Method 2:
 - Form a schema for each entity set with all local and inherited attributes

schema	attributes
person	ID, name, street, city
student	ID, name, street, city, tot_cred
employee	ID, name, street, city, salary

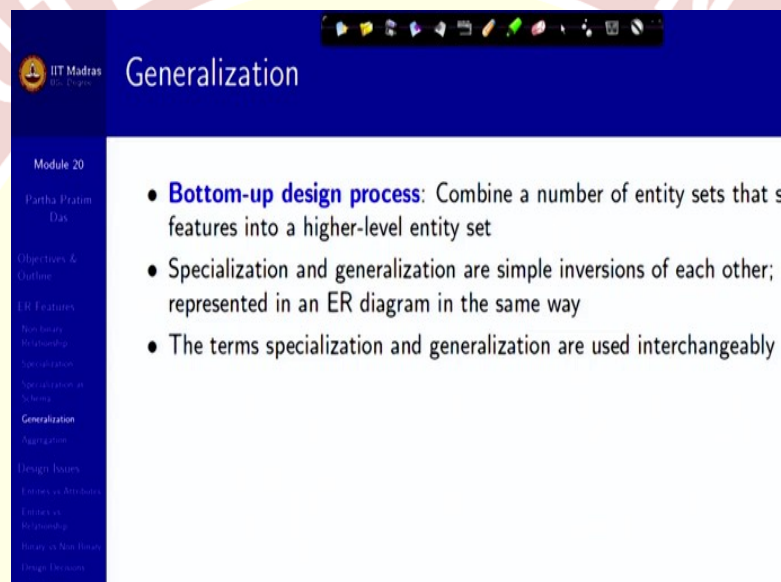
- Drawback: name, street and city may be stored redundantly for both students and employees

So, the other could be you do a flattened out design. That you simply just do the inheritance and borrow so person has everything. In student, the low level one, you take all the attributes of the person you are specializing from and at the, like this. So, the advantage here is, you do not have to traverse between relationships, between entities or between relations.

So, it is, it will be lot more efficient to do. But a little flip side is that if some student is also an employee or rather an employee is also a student, then there will be two entries here, naturally, which is, so there will be duplication of name, street, city. The higher level entity sets attributes will be duplicated for those cases.

So, in overlapping case, you will have some redundancy and potential anomaly. Now, this is the reality of the modeling. So, you cannot say which one is that way better. It depends somewhat on what your application, particular application is, how often you see that the join will be required or how potential is the threat of anomalies happening and whether you can take care of those through constraints or through application programming and decide on which method of translation to relational schema you would prefer.

(Refer Slide Time: 13:25)

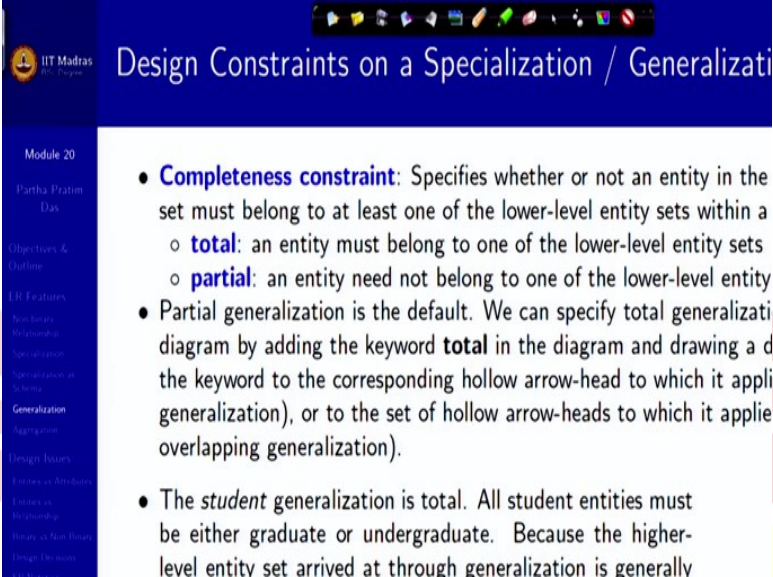


The screenshot shows a presentation slide from IIT Madras. The title is 'Generalization'. The slide is part of Module 20, presented by Partha Pratim Das. The left sidebar lists the following topics: Objectives & Outline, ER Features, Normalization, Specialization, Generalization (highlighted), Design Issues, Entity-Relationship, Database Design, and Design Decisions. The main content area contains three bullet points:

- **Bottom-up design process:** Combine a number of entity sets that share features into a higher-level entity set
- Specialization and generalization are simple inversions of each other; represented in an ER diagram in the same way
- The terms specialization and generalization are used interchangeably

This was a specialization coming from top to bottom. You can do the other. You can go from bottom to top, which is called the generalization. That is you have two entity sets which have significant stuff in common. So, take that common part, make it at a higher level entity set and go in the generalization way. Basically, one is the inversion of the other. Specialization and generalization one is, concept wise it is the same and they are used interchangeably, the terms are used interchangeably and you could keep on going like this.

(Refer Slide Time: 13:58)



Design Constraints on a Specialization / Generalization

- **Completeness constraint:** Specifies whether or not an entity in the set must belong to at least one of the lower-level entity sets within a
 - **total:** an entity must belong to one of the lower-level entity sets
 - **partial:** an entity need not belong to one of the lower-level entity sets
- Partial generalization is the default. We can specify total generalization diagram by adding the keyword **total** in the diagram and drawing a double line to the corresponding hollow arrow-head to which it applies (generalization), or to the set of hollow arrow-heads to which it applies (overlapping generalization).
- The *student* generalization is total. All student entities must be either graduate or undergraduate. Because the higher-level entity set arrived at through generalization is generally

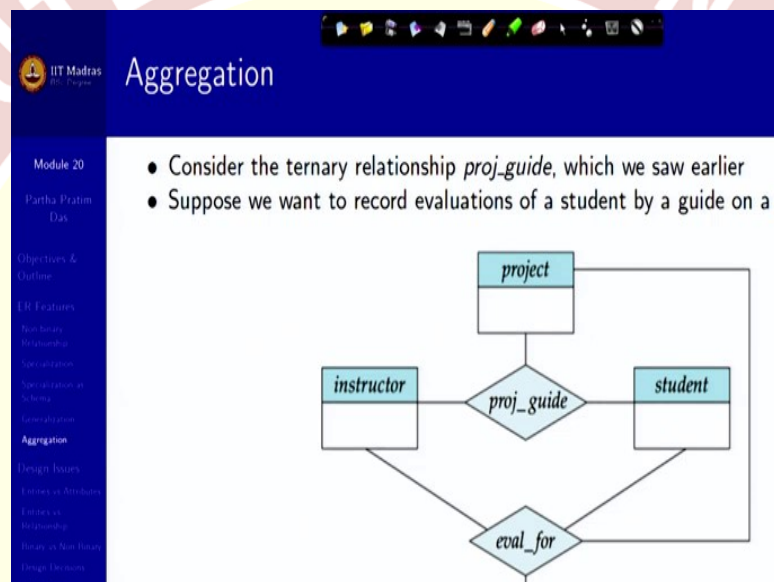
There may be a little difference in terms of the completeness constraints, because, for example, if you have done bottom-up, now, what will be the completeness in general? In general the completeness is partial. If you do not know, then you do not know whether a higher order entity set has all entities represented in any of the lower ordered entity sets. It may be, it may not be, so it is partial.

Now, it will be total if you have certain real world condition to think that it will have to be total. But if you do bottom-up generalization approach, then you are basically taking union. So, if you are taking union, then anything that you get on the higher order entity set has come from one of the lower order entity sets. So, it tends to be, if you do a generalized higher level entity set through the bottom-up approach, then it tends to be total, but it is not advisable to just go by that and do the analysis of the requirements to say if it is total.

So, for example, if I say that the students have, I am sorry, say I have two kinds of students, UG, and PG. Two other specialization of student, UG and PG and we will have attributes for that. Now, I can say the student is total in this specialization, because every student has to be either an undergraduate student or a postgraduate student, because that is how you become a student. So, just try to understand that it is not just thinking of a unification or generalization in the schema, it is the reality that you have to check on, is how do you become a student.

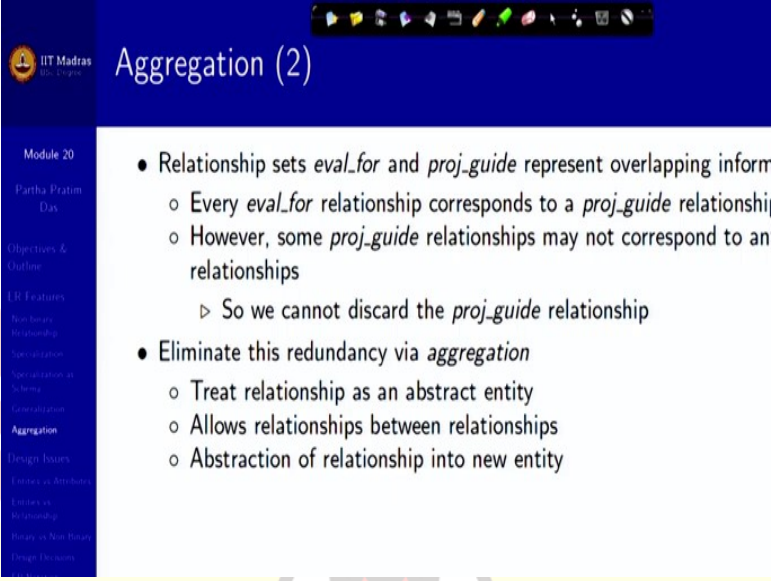
You become a student by either enrolling for the UG or enrolling for the PG. Otherwise, you are not a student. So, if somebody is a student, it becomes imperative that that person has to be either an undergraduate student or a postgraduate student. So, this relationship will be total in the aggregation. So, we can mark this by a dashed line and write total that is the notation that ER diagram specifies which says that this specialization goes total in that. So, that is a notion of total and partial. These are things that we may use subsequently.

(Refer Slide Time: 17:10)



Let us next talk about aggregation. Now, think about the project-guide ternary relationship. Now, we need another relationship which is the evaluation. So, the project will be evaluated. So, we introduce another relationship *eval_for*, which is between, again, between project, student and instructor.

(Refer Slide Time: 17:47)



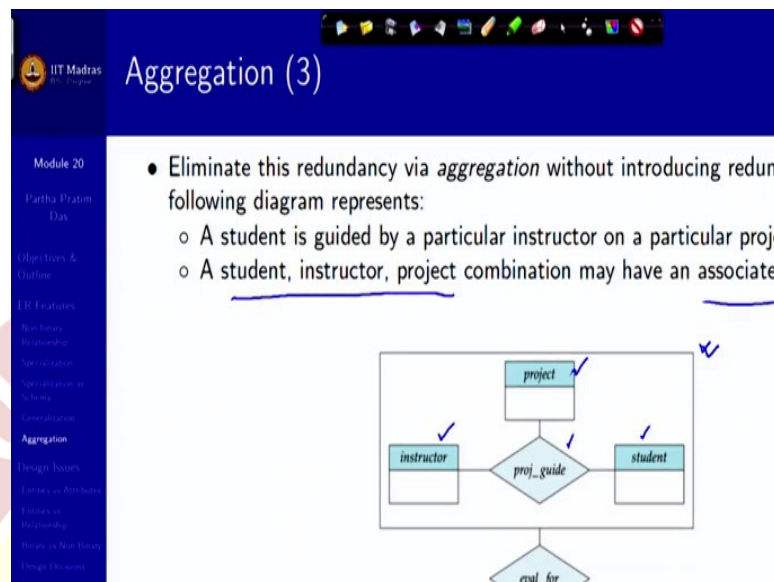
Aggregation (2)

- Relationship sets *eval_for* and *proj_guide* represent overlapping information
 - Every *eval_for* relationship corresponds to a *proj_guide* relationship
 - However, some *proj_guide* relationships may not correspond to an *eval_for* relationship
 - ▷ So we cannot discard the *proj_guide* relationship
- Eliminate this redundancy via *aggregation*
 - Treat relationship as an abstract entity
 - Allows relationships between relationships
 - Abstraction of relationship into new entity

So, you can see that both of these *eval_for* and *project_guide*, they have overlapping information, they have redundancy, because every *eval_for* relationship corresponds to a *project_guide* relationship, because it cannot be that I am evaluating xyz this triplet, whereas in the *project_guide* it is xyw that cannot happen. So, *eval_for* anyway corresponds to the *project_guide* relationship. But I cannot just have *eval_for* and remove *project_guide* relationship because there may be some projects which are not being evaluated. So, they will not have an *eval_for* entry.

So, this is a kind of redundancy which cannot be resolved at this level. So, what is introduced here is a concept of a abstraction of a relationship into an entity. That through this relationship the entire thing can you look at as an entity.

(Refer Slide Time: 19:05)



So, this is how you represent that. You are saying, just see carefully, this is, these are the things we had and we have put an enclosing box. This enclosing box says that this whole thing together, the project_guide is an entity as well, can be treated as an entity. If it is then for that I have an evaluation. So, now, it is not necessary that every project_guide entry has to have an evaluation. But I will have an evaluation for a particular project guide situation, project_guide entry, project_guide, which means basically student, instructor, project combination which will have an associated evaluation.

So, I am looking at it little differently than just having a decomposed view of two different ternary relations. This remove the redundancy, makes the whole concept clearer, and allows me to model in a better way.

(Refer Slide Time: 20:11)

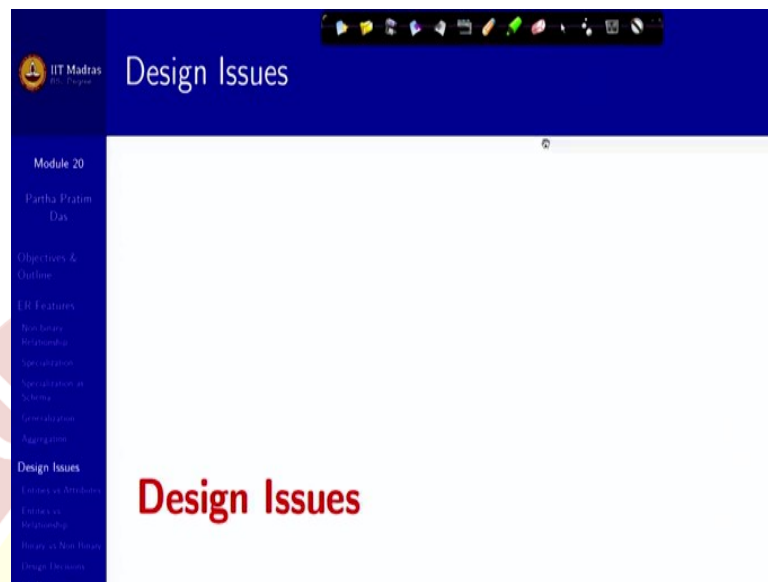
Representing Aggregation via Schema

- To represent aggregation, create a schema containing
 - Primary key of the aggregated relationship, ✓
 - The primary key of the associated entity set
 - Any descriptive attributes
- In our example:
 - The schema ~~text~~eval_for is:
eval_for (s_ID, project_id, i_ID, evaluation_id)
 - The schema proj_guide is redundant

So, when it comes to representation, the schema is, I am sorry, there is a typo here, I am sorry. This is not there. This text is not there. It is kind of a latex command which somehow has script in. So, the name is eval_for as we have seen. So, to represent the aggregation and create a schema containing the primary key of the aggregated relationship, because I will need that, I need the primary key of the associated entity sets that is evaluation and if I need any other additional descriptive words.

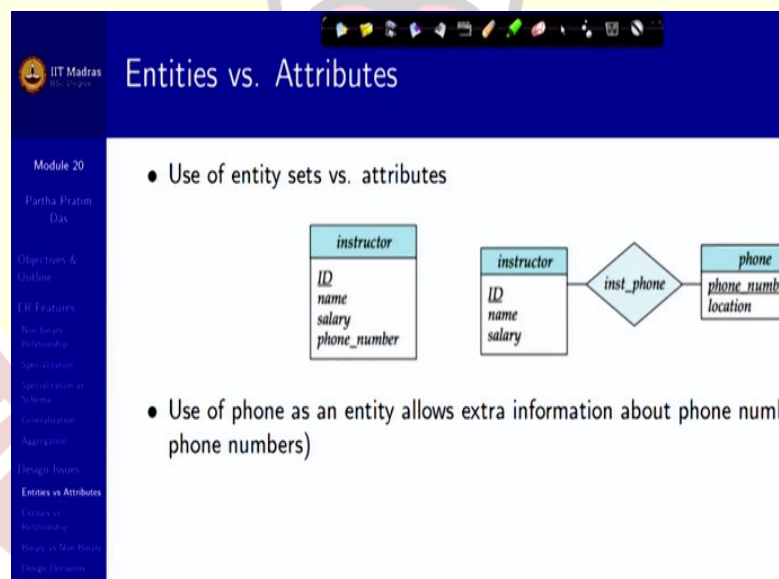
So, primary key of the project ID relationship are these three coming from three entity sets. And for the evaluation, this is the evaluation ID. There could be other auxiliary ones, which will. So, this whole eval_for will be represented in this way, so that the schema project guide can now be made redundant.

(Refer Slide Time: 21:42)



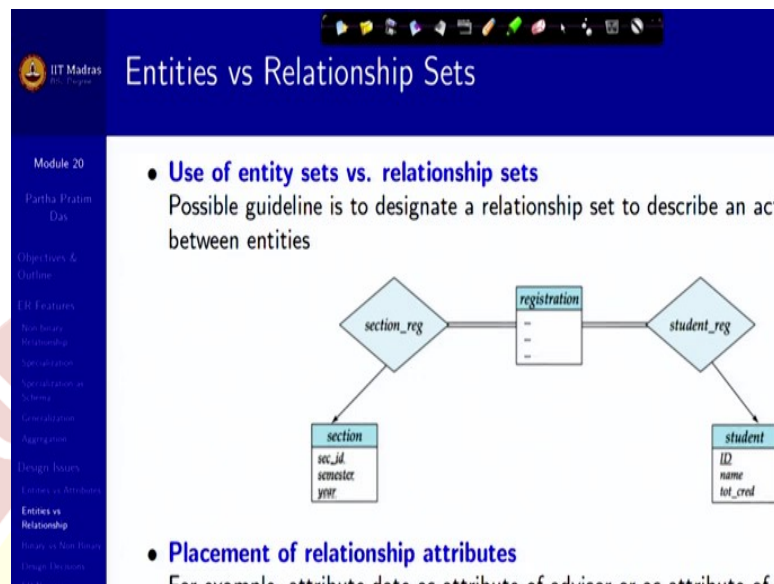
So, before we close, let us discuss a few design issues.

(Refer Slide Time: 21:48)



Obviously, there is always a question of what is an attribute and what is an entity? They are interchangeable in a certain way. For example, in phone number, the phone number could be an attribute. If it is an attribute, it is necessarily single valued. If you want it to be multi-valued, you will have to make it an entity and connect it through a relationship, as we have seen. So, these are considerations, which will be given.

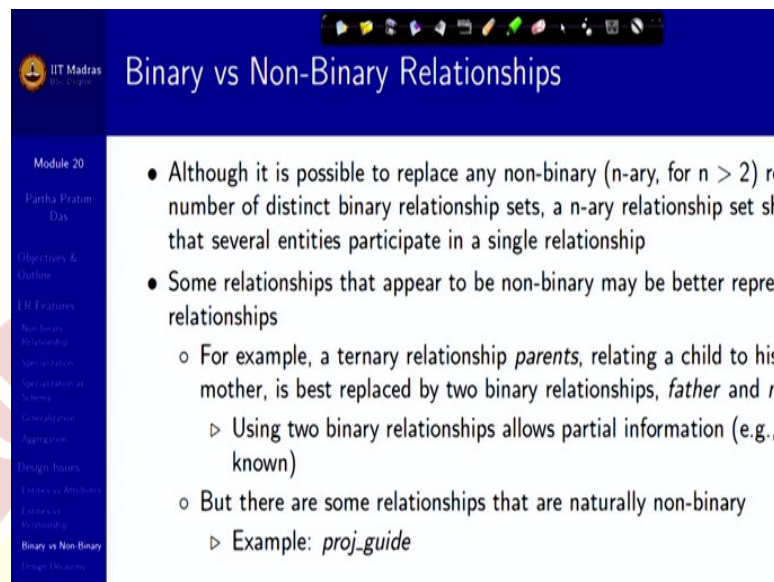
(Refer Slide Time: 22:13)



There is always a trade-off in terms of what is entities and what are relationships. We already have seen some of that earlier in terms of our instructor, course and student, department, instructor, student and department between them that whether we need those relationships or not. So, these kind of, so you will have to see that if things are total, for example, these are registration which is going through a registration, section registration and student registrations.

Section registrations gives the registration information of the section. Student registration gives the information to the student. Do you really need these relationships or you can just convert it into a single entity relating the section with the student. So, these are considerations which we will go through.

(Refer Slide Time: 23:16)



IIT Madras
Module 20
Partha Pratim Das
Objectives & Outline
ER Features
Non-Binary Relationships
Normalization in Schema
Generalization
Aggregation
Design Issues
Conceptual Ambiguity
Database Normalization
Binary vs Non-Binary
Design Decisions

Binary vs Non-Binary Relationships

- Although it is possible to replace any non-binary (n -ary, for $n > 2$) re number of distinct binary relationship sets, a n -ary relationship set sh that several entities participate in a single relationship
- Some relationships that appear to be non-binary may be better repres relationships
 - For example, a ternary relationship *parents*, relating a child to his mother, is best replaced by two binary relationships, *father* and *n*
 - ▷ Using two binary relationships allows partial information (e.g., known)
 - But there are some relationships that are naturally non-binary
 - ▷ Example: *proj_guide*

The other, as I said that, most relations in the database design are binary. And, but there are situations when you have non-binary relationships. It may be more natural to think about it in that way. For example, you have the parents relationship. Parents relationship is ternary, because you have two parents. So, there is a person, there is a father, there is a mother. So, it is a ternary, three way relationship.

But this kind of a relationship is very easily decomposable in terms of binary relationship. I will have two relationships, because they do not, I mean, there is no that way relationship between father and mother. The relationship is between the person and the father, person and the mother. So, it can be two independent relationship, that two binary relationship we saw.

But there are situations like the *project_guide*, where it is naturally non-binary. You cannot have a relation between instructor and project or instructor and student, student and project. Those, none of them represent the reality of instructor, student, project being together. So, in those cases, it becomes difficult to, I think I skipped one.

(Refer Slide Time: 24:46)

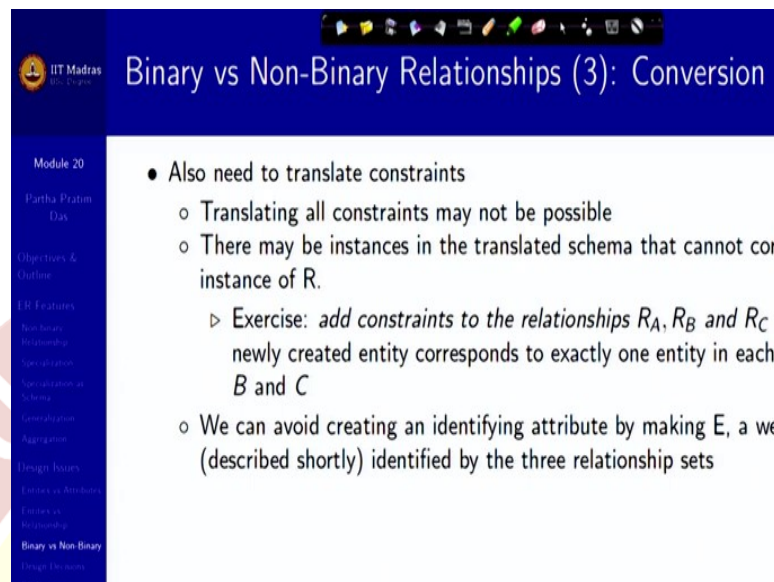
Binary vs Non-Binary Relationships (2): Conversion

- In general, any non-binary relationship can be represented using binary creating an artificial entity set.
 - Replace R between entity sets A, B and C by an entity set E, and sets:
 1. R_A , relating E and A
 2. R_B , relating E and B
 3. R_C , relating E and C
 - Create an identifying attribute for E and add any attributes of R
 - For each relationship (a_i, b_i, c_i) in R, create
 - a) a new entity e_i in the entity set E
 - b) add (e_i, a_i) to R_A
 - c) add (e_i, b_i) to R_B
 - d) add (e_i, c_i) to R_C

So, here is a situation, so these are some of the approaches of analyzing or representing through alternate. So, here is what I have here is a ternary relationship between A, B, C. What do you do? You can create, you can represent this in terms of binary relationship by introducing a new entity set E with an identifying attribute. What is E? E just has a primary key. It could be a synthetic one. So, you have E sitting here, coming in here in place of where R was and you have now three decomposed binary relation R_A , R_B , R_C .

So, here what did you have, you had (a_i, b_i, c_i) the triplet in this case. So, now you add (e_i, a_i) , (e_i, b_i) , (e_i, c_i) respectively to here, here and here. So, using e_i , you will be able to recombine and get the triplet information back. And using the triplet you will be able to go from the ternary to the decomposed binary relation. So, it's information preserving. It has the same information.

(Refer Slide Time: 26:28)



IIT Madras
Module 20
Partha Pratim Das
Objectives & Outline
ER Features
Non-binary Relationships
Specialization
Generalization
Aggregation
Design Issues
Entity vs. Attributes
Entity vs. Relationship
Binary vs Non Binary
Design Decisions

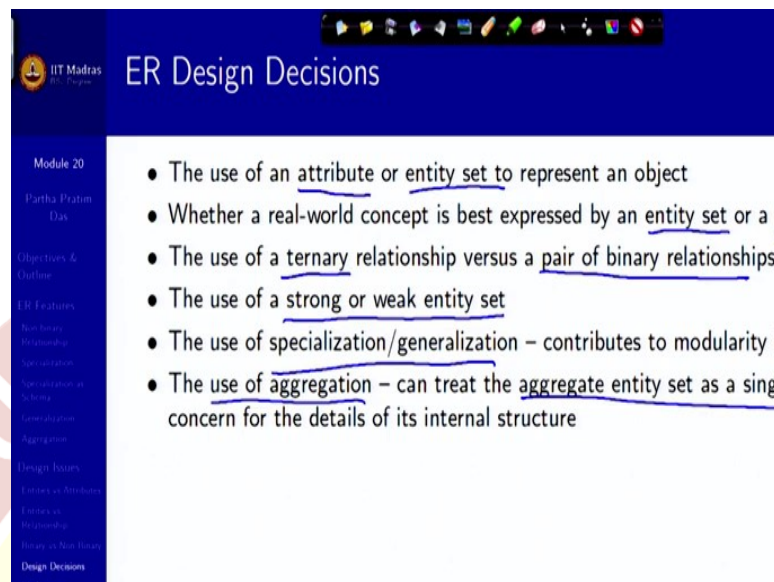
Binary vs Non-Binary Relationships (3): Conversion

- Also need to translate constraints
 - Translating all constraints may not be possible
 - There may be instances in the translated schema that cannot correspond to an instance of R .
 - ▷ Exercise: add constraints to the relationships R_A , R_B and R_C to ensure that the newly created entity corresponds to exactly one entity in each of B and C
 - We can avoid creating an identifying attribute by making E , a weak entity (described shortly) identified by the three relationship sets

But this is not all. What you need is also need to translate the constraints. And translating all constraints may not be easy, because when you are in a binary relation you know the information about two entities. So, if you have constraints which actually involve all the three entities that cannot be directly enforced on the decomposed binary relations, so you have to change the constraints and think of how you can check for what you are needing to do.

Further is, we can also avoid creating the identifying attribute by making E a weak entity set. So, it is a weak entity set which has no attribute. So, this is, because that is a basic role this is just to relate.

(Refer Slide Time: 27:39)



IIT Madras
ER Design Decisions

Module 20
Partha Pratim Das

Objectives & Outline
ER Features
Non-binary Relationships
Specialization & Generalization
Aggregation
Design Issues
Entity vs. Attribute
Entity vs. Relationship
Design in Non-binary
Design Decisions

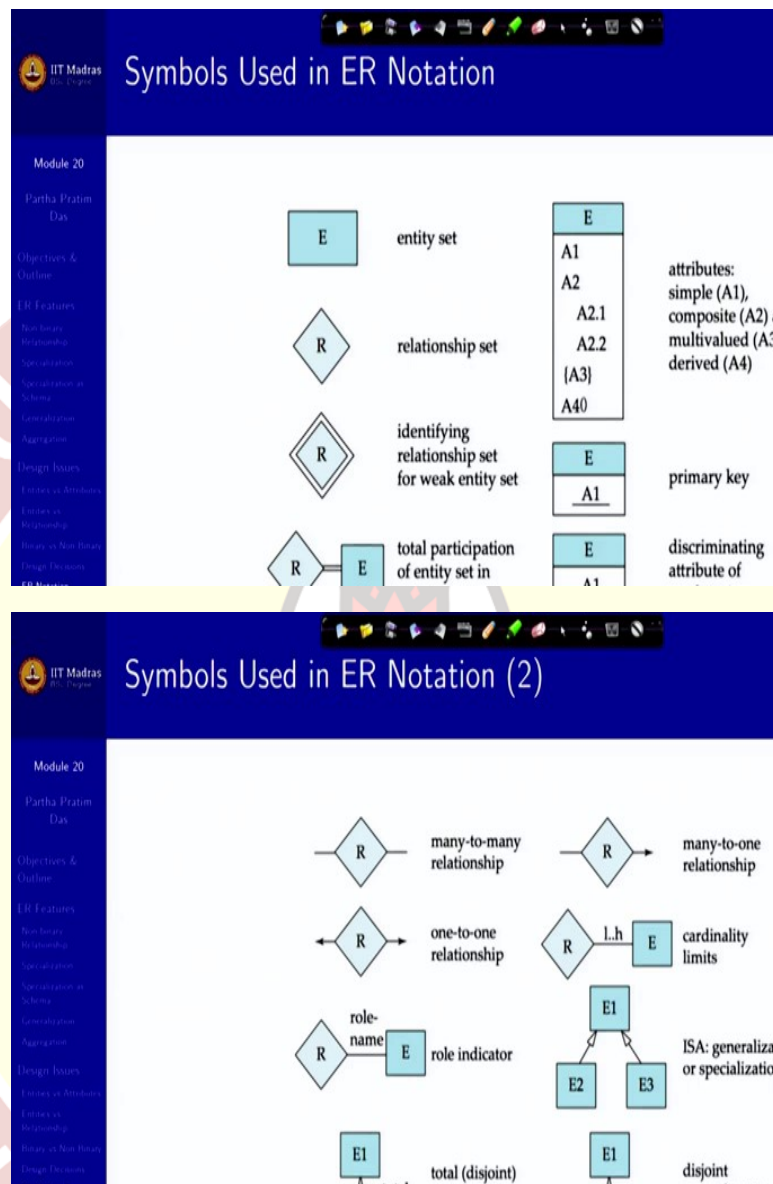
- The use of an attribute or entity set to represent an object
- Whether a real-world concept is best expressed by an entity set or a relationship set
- The use of a ternary relationship versus a pair of binary relationships
- The use of a strong or weak entity set
- The use of specialization/generalization – contributes to modularity in design
- The use of aggregation – can treat the aggregate entity set as a single concern for the details of its internal structure

Now, these are some of the key design decisions you need to take for entity relationship model, entity relationship design, use of attribute or entity set for representing objects. What should be good as entity set and what should be good as relationship set. And there is no universal rule for them, maybe some thumb rules, but it is more of judgment, experience and quality of design. And often, designing this way or that way may give two equally good alternates. The use of ternary relationship versus pair of binary relationships, we have talked about this.

Use of strong and weak entity sets. These are the key design areas. So, when you do the design, focus on these and ask yourself, am I using this, is it okay or should I have used the other. Use of specialization, generalization, which significantly helps in the modularity of the design, because you are not cluttering all things of student and all things of employees with everything a person, student side gone, employee side separate, person's general side separate, much more modular in that way.

Use of aggregation, where aggregated entity can be, so it is kind of a hierarchical conceptualization that you are doing. You are aggregating stuff together and creating a new concept of an entity through the relationship interactions that are happening. So, these are some of the key considerations of entity relationship logical design. So, as we go through the tutorial and exercise you will slowly start getting more insights into that.

(Refer Slide Time: 29:35)

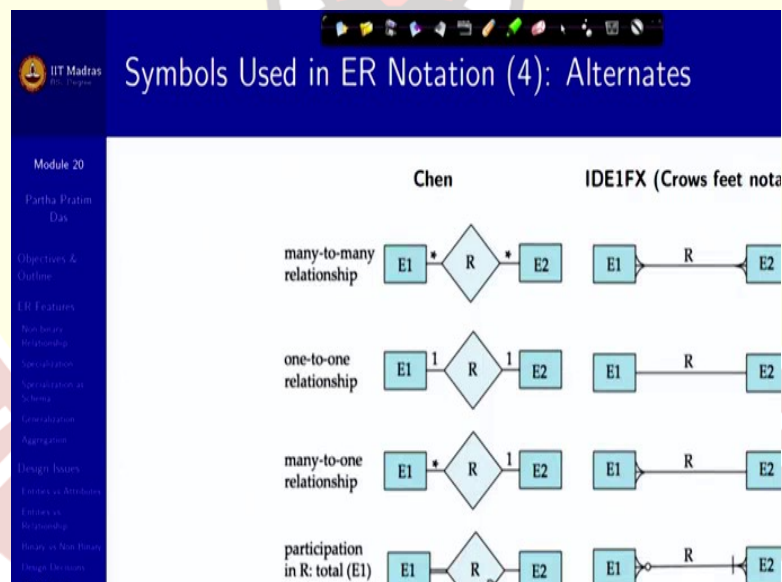
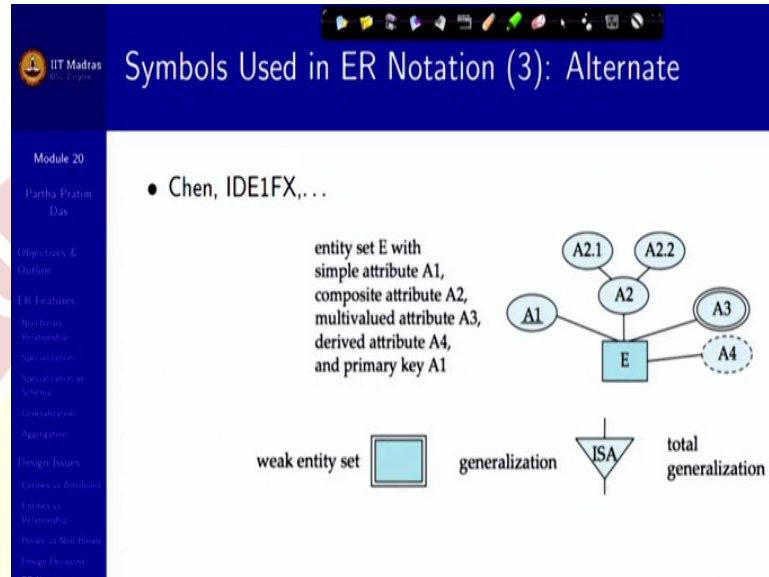


Remaining couple of slides are for your reference where we have put the symbols that are used in ER notation. These are significantly from UML, but doesn't mean that you have to go and open up UML books. Don't try to do that. But these are the very, these are not only, it is not that that we are following this notation or this book is following the notation.

The advantage of this notation is this notation is kind of universal and standardized, internationally standardized. So, you use it in any company in any organization it should be recognizable in the other. So, entity set relationships, all of these that we have seen. I mean, we

have tried to cover all the, meaning of all the notation and the consequences of that. So, you should be able to see through this.

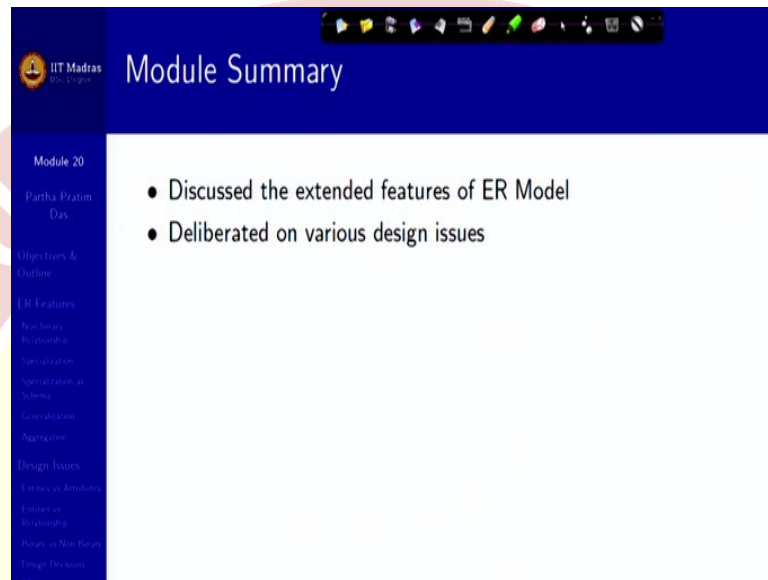
(Refer Slide Time: 30:25)



Of course, here there is an alternate. You can see that like a blooming tree, you have E is entity and A1, A2 are attributes and then they are shown in terms of their composition. The ISA is shown in a different way and so on. So, these used to be the earlier representation for decades this representation has been used, but, so you will see that in several places in books and Internet, but do not use it afresh because people have moved down from here and so on.

So, these are the alternates that exist in other system the Chen so on. So, these are, I am not, we are not suggesting that you use it, but we have given it here so that if you get some design, so that you will be able to understand okay it is in this notation. So, read up on that notation.

(Refer Slide Time: 31:29)



IIT Madras
Module Summary

- Discussed the extended features of ER Model
- Deliberated on various design issues

Module 20
Partha Pratim Das

Objectives & Outline

ER Features

- New Entity Relationship
- Generalization
- Specialization
- Aggregation

Design Issues

- Contextual Ambiguity
- Entity vs. Relationship
- Design in New Design
- Design Decision

So, we have, with this we are completing the discussion on the extended and total features of entity relationship model as we need. We have talked about a few design issues. We will continue to discuss this and thank you very much for your attention. And we will take up new aspects of database design in the subsequent modules.